HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN

BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: IOT VÀ ỨNG DỤNG
MÃ HỌC PHẦN: INT14149

ĐỀ TÀI
HỆ THỐNG SMART HOME GIÁM SÁT VÀ ĐIỀU KHIỂN THIẾT BỊ DỰA TRÊN CẢM BIẾN MÔI TRƯỜNG

Giảng viên hướng dẫn: TS Nguyễn Quốc Uy

HỌC KỲ 1 NĂM HỌC 2026-2027

MỤC LỤC

MỤC LỤC *2*

Danh mục các hình Vẽ *4*

Danh mục các bảng biểu *4*

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI *5*

1.1 Bối cảnh và lý do chọn đề tài **5**

1.2 Mục tiêu đề tài **5**

1.3 Phạm vi đề tài **5**

1.4 Kết quả cần đạt **6**

CHƯƠNG 2. KIẾN TRÚC VÀ CÔNG NGHỆ HỆ THỐNG *6*

2.1 Kiến trúc tổng thể **6**

2.1.1 Khối phần cứng ESP8266 8

2.1.2 Khối cảm biến 9

2.1.3 MQTT Broker và giao thức MQTT 9

2.1.4 Backend 10

2.1.5 Database 11

2.1.6 Frontend 11

CHƯƠNG 3. THIẾT KẾ CƠ SỞ DỮ LIỆU *12*

3.1 Bảng user **12**

3.2 Bảng iot_node **14**

3.3 Bảng sensor **14**

3.4 Bảng sensor_data **15**

3.5 Bảng actuator **16**

3.6 Bảng action_history **17**

3.7 Quan hệ giữa các bảng **18**

CHƯƠNG 4. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG *18*

4.1 Yêu cầu hệ thống **18**

4.1.1 Yêu cầu chức năng 18

4.1.2 Yêu cầu phi chức năng 18

4.2 Mô hình Use Case và Sequence Diagram **18**

4.2.1 UC01 - Đăng nhập hệ thống 19

4.2.2 Màn hình Đăng nhập 21

4.2.3 UC02 - Xem thông số môi trường hiện tại 21

4.2.4 Màn hình Dashboard 25

4.2.5 UC03 - Xem lịch sử dữ liệu cảm biến 25

4.2.6 Màn hình Lịch sử cảm biến 28

4.2.7 UC04 – Điều khiển bật/tắt thiết bị 28

4.2.8 UC05 - Xem lịch sử điều khiển thiết bị 31

4.2.9 Màn hình lịch sử điều khiển thiết bị 33

4.2.10 Màn hình Profile 34
CHƯƠNG 5. CÀI ĐẶT BACKEND VÀ CƠ SỞ DỮ LIỆU 35
5.1 Tổ chức mã nguồn 35
5.2 Xử lý lệnh bật tắt 35
5.3 Lưu trữ và xác thực 37
5.4 Kiểm tra API 37
CHƯƠNG 6. THIẾT KẾ VÀ TÀI LIỆU API 37
6.1 Danh sách API 37
6.2 Mô tả các API 38
6.3 Luồng giao tiếp API và MQTT 39
CHƯƠNG 7. TÀI LIỆU VÀ MÃ NGUỒN 39

DANH MỤC CÁC HÌNH VẼ

Hình 1 - Kiến trúc tổng thể 8
Hình 2 - Thiết kế DB 12
Hình 3 - Sequence Diagram UC1 20
Hình 4 - Màn hình Đăng nhập 21
Hình 5 - Màn hình Đăng ký tài khoản 21
Hình 6 - Sequence Diagram UC2 24
Hình 7 - Màn hình Dashboard 25
Hình 8 - Sequence Diagram UC3 28
Hình 9 - Màn hình Lịch sử cảm biến 28
Hình 10 - Sequence Diagram UC4 31
Hình 11 - Sequence Diagram UC5 33
Hình 12 - Màn hình Lịch sử hoạt động 34
Hình 13 - Màn hình Profile 34
Hình 14 - ApiController nhận yêu cầu điều khiển và trả HTTP 202 35
Hình 15 - RoomService lưu lệnh PENDING rồi gửi MQTT 36
Hình 16 - TelemetryService đối soát phản hồi từ ESP8266 36
Hình 17 - SmartRoomRepository truy vấn lịch sử điều khiển 37

DANH MỤC CÁC BẢNG BIỂU

Bảng 1 - Cảm biến 9
Bảng 2 - Các topic 10
Bảng 3 - Tổng quan DB 11
Bảng 4 - Bảng DB User 13
Bảng 5 - Bảng DB iot-node 14
Bảng 6 - Bảng DB sensor 14
Bảng 7 - Bảng DB sensor data 15
Bảng 8 - Bảng DB actuator 16
Bảng 9 - Bảng DB action history 17
Bảng 10 - Danh sách UC 19
Bảng 11 - Danh sách các API 38

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Bối cảnh và lý do chọn đề tài

Trong các không gian học tập và sinh hoạt, việc theo dõi nhiệt độ, độ ẩm và mức ánh sáng giúp người dùng đánh giá nhanh điều kiện môi trường. Tuy nhiên, nếu dữ liệu chỉ được quan sát trực tiếp tại thiết bị thì khó lưu trữ, tra cứu và điều khiển từ xa. Dự án Smart Room IoT được xây dựng để kết nối cảm biến, thiết bị mô phỏng và ứng dụng web trong một quy trình thống nhất, qua đó minh họa đầy đủ chuỗi thu thập dữ liệu, truyền thông, lưu trữ và tương tác người dùng.

1.2 Mục tiêu đề tài

Thu thập và hiển thị dữ liệu nhiệt độ, độ ẩm và cường độ ánh sáng từ hệ thống dựa trên ESP8266.

Lưu dữ liệu cảm biến để người dùng xem giá trị mới nhất và lịch sử theo thời gian.

Cho phép người dùng bật/tắt hai thiết bị logic Fan và Light trên giao diện web; trong mô hình thử nghiệm, hai LED được dùng để mô phỏng trạng thái thiết bị.

Sử dụng MQTT cho giao tiếp bất đồng bộ giữa Backend và ESP8266; sử dụng REST API cho giao tiếp giữa Frontend và Backend.

Theo dõi trạng thái mong muốn, trạng thái phần cứng xác nhận và lịch sử xử lý lệnh để bảo đảm khả năng truy vết.

1.3 Phạm vi đề tài

- Phần cứng
  - Vi điều khiển ESP8266.
  - Cảm biến DHT11 để đo nhiệt độ và độ ẩm.
  - Cảm biến ánh sáng analog nối chân A0 của ESP8266; số đo nằm trong khoảng ADC 0–1023.
  - Hai LED ở D6 và D7 đang mô phỏng quạt và đèn; ba LED ở D0, D1, D5 được lắp dự phòng.
- Phần mềm
  - Lập trình C/C++ trên ESP8266 để thu thập dữ liệu và thực hiện logic điều khiển.
  - Thiết kế cơ sở dữ liệu để lưu trữ dữ liệu cảm biến và lịch sử điều khiển.
  - Phát triển ứng dụng web cho phép:
  - Hiển thị dữ liệu cảm biến theo thời gian thực.
  - Hiển thị trạng thái hoạt động của các thiết bị.

- Điều khiển bật/tắt thiết bị thủ công.
- Theo dõi lịch sử dữ liệu và lịch sử điều khiển.

1.4 Kết quả cần đạt

Kết quả của đề tài là một mô hình có khả năng nhận dữ liệu cảm biến, lưu trữ và hiển thị trên Dashboard; cung cấp lịch sử dữ liệu; gửi lệnh ON/OFF đến ESP8266 qua MQTT; ghi nhận phản hồi thực tế; và lưu lịch sử thao tác. Tài liệu thiết kế gồm Use Case, mô hình dữ liệu, sequence diagram, thiết kế giao diện và bộ yêu cầu API trên Postman.

CHƯƠNG 2. KIẾN TRÚC VÀ CÔNG NGHỆ HỆ THỐNG

2.1 Kiến trúc tổng thể

Hệ thống Smart Room IoT được thiết kế theo kiến trúc nhiều lớp, bao gồm các thành phần chính: Người dùng, Frontend Web, Backend, Cơ sở dữ liệu, MQTT Broker và khối phần cứng ESP8266. Kiến trúc này giúp tách biệt phần giao diện, xử lý nghiệp vụ, lưu trữ dữ liệu và giao tiếp với thiết bị IoT.

Dựa trên các chức năng của hệ thống, kiến trúc tổng thể gồm **6 thành phần chính**:

1. **Người dùng (User):** tương tác với hệ thống thông qua giao diện Web để theo dõi dữ liệu cảm biến, kiểm tra trạng thái thiết bị và thực hiện các thao tác bật hoặc tắt Fan, Light.
2. **Frontend Web:** cung cấp giao diện Dashboard, hiển thị nhiệt độ, độ ẩm, cường độ ánh sáng, trạng thái kết nối của ESP8266 và trạng thái của các thiết bị điều khiển. Frontend gửi yêu cầu đến Backend thông qua REST API và không truy cập trực tiếp vào cơ sở dữ liệu hoặc phần cứng.
3. **Backend:** là thành phần trung tâm của hệ thống, có nhiệm vụ tiếp nhận yêu cầu từ Frontend, xử lý nghiệp vụ, truy vấn cơ sở dữ liệu và giao tiếp với MQTT Broker. Backend đồng thời đóng vai trò MQTT Client, có thể publish lệnh điều khiển và subscribe các topic dữ liệu hoặc trạng thái thiết bị.
4. **Cơ sở dữ liệu (Database):** lưu trữ thông tin người dùng, nút IoT, cảm biến, dữ liệu cảm biến, thiết bị chấp hành và lịch sử điều khiển. Cơ sở dữ liệu gồm 6 bảng chính: user, iot_node, sensor, sensor_data, actuator và action_history.
5. **MQTT Broker:** đóng vai trò trung gian truyền bản tin giữa Backend và ESP8266 theo mô hình Publish/Subscribe. MQTT Broker tiếp nhận bản tin từ bên publish và chuyển bản tin đến các bên đã subscribe topic tương ứng.
6. ESP8266 đọc DHT11 ở D2 và cảm biến ánh sáng analog ở A0. Hai LED đang điều khiển là D6 (Quạt) và D7 (Đèn LED); D0, D1, D5 là chân dự phòng.

Các thành phần trong hệ thống giao tiếp với nhau theo hai hình thức chính:

- **REST API:** được sử dụng trong giao tiếp giữa Frontend và Backend. Frontend gửi các HTTP Request để lấy dữ liệu hoặc thực hiện thao tác điều khiển; Backend xử lý và trả về kết quả dưới dạng JSON.

- **MQTT:** được sử dụng trong giao tiếp bất đồng bộ giữa Backend và ESP8266. Hai bên không gửi bản tin trực tiếp cho nhau mà trao đổi thông qua MQTT Broker và các topic đã được cấu hình.

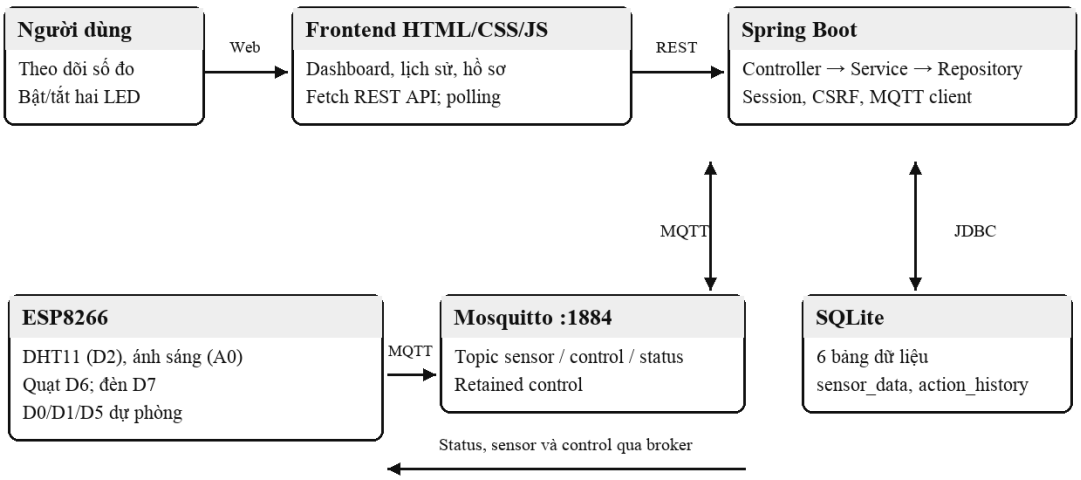
Hệ thống có ba luồng hoạt động chính:

- ESP8266 gửi nhiệt độ, độ ẩm và giá trị ánh sáng ADC lên ba topic MQTT. Backend nhận bản tin, kiểm tra miền giá trị và lưu vào SQLite. Frontend lấy dữ liệu từ GET /api/state.
- **Luồng điều khiển thiết bị:** người dùng thực hiện thao tác bật hoặc tắt Fan/Light trên Frontend. Frontend gửi yêu cầu đến Backend thông qua REST API. Backend cập nhật desired_state, tạo bản ghi lịch sử điều khiển và publish lệnh lên MQTT Broker. ESP8266 nhận lệnh, thay đổi trạng thái GPIO và điều khiển LED tương ứng.
- **Luồng phản hồi trạng thái:** sau khi thực hiện lệnh, ESP8266 publish trạng thái thực tế của thiết bị dưới dạng reported_state lên MQTT Broker. Backend nhận phản hồi, cập nhật trạng thái thiết bị và lịch sử điều khiển trong cơ sở dữ liệu. Frontend lấy trạng thái mới nhất từ Backend để hiển thị cho người dùng.

Đối với lệnh điều khiển, Backend sử dụng cơ chế retained message với retain = true. MQTT Broker sẽ lưu lại lệnh mới nhất trên control topic. Nếu ESP8266 đang ngoại tuyến tại thời điểm lệnh được gửi, thiết bị vẫn có thể nhận lại lệnh khi kết nối và subscribe topic trở lại.

Kiến trúc trên được thiết kế để đáp ứng các chức năng chính của hệ thống gồm:

- Hiển thị dữ liệu nhiệt độ, độ ẩm và cường độ ánh sáng trên Dashboard.
- Hiển thị lịch sử dữ liệu cảm biến.
- Điều khiển bật hoặc tắt Fan và Light.
- Theo dõi trạng thái kết nối Online/Offline của ESP8266.
- Phân biệt trạng thái mong muốn desired_state và trạng thái thực tế reported_state.
- Lưu và hiển thị lịch sử điều khiển thiết bị.
- Lọc lịch sử theo thiết bị, hành động, trạng thái và thời gian.
- Quản lý thông tin tài khoản và Profile của người dùng.
- Duy trì khả năng đồng bộ trạng thái thiết bị sau khi ESP8266 kết nối lại.



Trạng thái SUCCESS chỉ xuất hiện khi ESP8266 xác nhận lệnh.

Hình 1 - Kiến trúc tổng thể

2.1.1 *Khối phần cứng ESP8266*

Firmware ESP8266 viết bằng C/C++, dùng Wi-Fi và MQTT để truyền dữ liệu. Mạch đọc DHT11 ở D2, đầu vào ánh sáng A0 và điều khiển hai LED tại D6, D7.

- Các chức năng chính của ESP8266 gồm:
  - Kết nối mạng Wi-Fi và duy trì kết nối với MQTT Broker.
  - Đọc dữ liệu nhiệt độ, độ ẩm và ánh sáng từ các cảm biến tương ứng.
  - Publish riêng từng giá trị cảm biến lên smartroom/sensor/temperature, humidity và light sau khoảng 3 giây.
  - Subscribe các topic điều khiển của Fan và Light.
  - Nhận lệnh ON/OFF, kiểm tra payload và thay đổi mức GPIO tương ứng.
  - Điều khiển hai LED dùng để mô phỏng trạng thái bật/tắt của Fan và Light.
  - Publish reported_state sau khi thay đổi GPIO để Backend ghi nhận trạng thái thực tế.
  - Tự động kết nối lại Wi-Fi hoặc MQTT và đăng ký lại topic khi kết nối bị gián đoạn.

LED ở D6 mô phỏng quạt; LED ở D7 mô phỏng đèn. Ba LED nối D0, D1, D5 được khởi tạo ở mức OFF để dự phòng và chưa có API hoặc topic điều khiển.

Sau khi thực hiện lệnh, ESP8266 công bố trạng thái thực tế để Backend cập nhật reported_state. Cơ chế phản hồi này giúp hệ thống không đồng nhất một cách giả định trạng thái mong muốn với trạng thái đã thực hiện.

2.1.2 *Khối cảm biến*

DHT11 đo nhiệt độ và độ ẩm. Cảm biến ánh sáng được đọc qua ADC A0; hệ thống lưu giá trị 0–1023 với đơn vị ADC, không quy đổi thành lux.

Nhóm dữ liệu	Đại lượng	Giá trị và đơn vị	Mục đích sử dụng
Môi trường	Nhiệt độ	°C, DHT11	Theo dõi hiện tại và lịch sử
Môi trường	Độ ẩm	%, DHT11	Theo dõi hiện tại và lịch sử
Môi trường	Ánh sáng	ADC 0–1023, A0	Theo dõi hiện tại và lịch sử

Bảng 1 - Cảm biến

2.1.3 *MQTT Broker và giao thức MQTT*

MQTT được sử dụng làm kênh giao tiếp giữa Backend và ESP8266. Giao thức hoạt động theo mô hình Publish/Subscribe: bên gửi publish bản tin lên một topic; MQTT Broker tiếp nhận và chuyển bản tin đến các client đã subscribe topic đó. Cơ chế này phù hợp với thiết bị IoT có tài nguyên hạn chế và hỗ trợ giao tiếp bất đồng bộ.

Trong hệ thống, các topic được chia theo mục đích thay vì dùng chung một kênh. Nhóm topic dữ liệu cảm biến truyền số liệu từ ESP8266 đến Backend. Nhóm topic điều khiển truyền desired_state từ Backend đến ESP8266. Nhóm topic trạng thái truyền reported_state từ ESP8266 về Backend. Tên topic, QoS và cấu trúc payload phải được thống nhất trong contract thực tế của dự án.

Nhóm topic	Chiều truyền	Nội dung chính
smartroom/sensor/{temperature,humidity,light}	ESP8266 → Backend	Giá trị đo dạng số
smartroom/{fan,light}/control	Backend → ESP8266	ON/OFF, retained
smartroom/{fan,light}/status	ESP8266 → Backend	ON/OFF thực tế

Bảng 2 - Các topic

Đối với lệnh điều khiển, Backend có thể publish bản tin với cờ retained. Broker lưu bản tin điều khiển mới nhất của mỗi topic và gửi lại bản tin đó khi ESP8266 kết nối hoặc subscribe lại. Nhờ vậy, nếu thiết bị đang offline tại thời điểm người dùng gửi lệnh, trạng thái mong muốn gần nhất vẫn có thể được chuyển đến thiết bị sau khi kết nối được khôi phục.

Retained message chỉ đại diện cho trạng thái mong muốn gần nhất trên topic, không thay thế lịch sử điều khiển trong Database. Lịch sử từng yêu cầu vẫn được lưu tại action_history. Khi thay đổi cấu trúc topic hoặc loại bỏ thiết bị, hệ thống cũng cần có cơ chế xóa retained message cũ để tránh áp dụng nhầm lệnh đã lỗi thời.

2.1.4 *Backend*

Backend là lớp xử lý nghiệp vụ trung tâm và là điểm kết nối giữa Frontend, Database và MQTT Broker. Frontend gửi yêu cầu qua REST API; Backend xác thực và kiểm tra dữ liệu, thực hiện truy vấn cần thiết, sau đó trả kết quả theo mã trạng thái HTTP phù hợp.

Các trách nhiệm chính của Backend gồm:

Cung cấp REST API phục vụ đăng nhập, hiển thị cảm biến, trạng thái thiết bị và lịch sử thao tác.

Kiểm tra tham số, payload và quyền truy cập trước khi xử lý yêu cầu.

Nhận bản tin cảm biến từ MQTT, chuẩn hóa và lưu dữ liệu vào Database.

Tiếp nhận yêu cầu bật/tắt Fan hoặc Light từ Frontend.

Cập nhật desired_state và tạo bản ghi action_history với trạng thái PENDING trước khi publish lệnh.

Publish lệnh điều khiển MQTT với retained message theo contract của dự án.

Nhận reported_state từ ESP8266, cập nhật trạng thái thực tế và chuyển action_history từ PENDING sang SUCCESS khi xác nhận phù hợp.

Cung cấp dữ liệu mới nhất và dữ liệu lịch sử cho Frontend thông qua REST API.

Yêu cầu điều khiển là một quá trình bất đồng bộ. Vì vậy, sau khi đã lưu yêu cầu và publish lệnh, Backend có thể trả HTTP 202 Accepted cùng desiredState và commandStatus = PENDING. Phản hồi này có nghĩa là hệ thống đã tiếp nhận lệnh nhưng chưa khẳng định phần cứng đã thực hiện thành công. Chỉ khi nhận được reported_state phù hợp từ ESP8266, Backend mới cập nhật action_history.status = SUCCESS.

Việc phân biệt các trường trạng thái là bắt buộc: desired_state thể hiện mong muốn của người dùng; reported_state thể hiện trạng thái thực tế của LED; action_history.status thể hiện tiến trình xử lý lệnh với PENDING hoặc SUCCESS.

2.1.5 *Database*

Database là lớp lưu trữ lâu dài của hệ thống. Backend là thành phần duy nhất trực tiếp thực hiện truy vấn và cập nhật dữ liệu; Frontend không chứa thông tin kết nối Database và chỉ trao đổi qua REST API.

Nhóm dữ liệu	Nội dung lưu trữ	Vai trò
user	Tài khoản, BCrypt, hồ sơ, vai trò	Xác thực người dùng
iot_node	Thông tin ESP8266, last_seen	Theo dõi kết nối
sensor	Loại, đơn vị và MQTT topic	Mô tả nguồn đo
sensor_data	Giá trị REAL và thời gian UTC	Lịch sử cảm biến
actuator	LIGHT/FAN, desired và reported	Trạng thái hai LED
action_history	Lệnh, PENDING/SUCCESS/FAILED, phản hồi	Nhật ký điều khiển

Bảng 3 - Tổng quan DB

2.1.6 *Frontend*

Frontend là lớp giao diện tương tác trực tiếp với người dùng và được phát triển dưới dạng ứng dụng Web. Frontend trình bày dữ liệu do Backend cung cấp, tiếp nhận thao tác bật/tắt và gửi yêu cầu theo các REST API đã định nghĩa. Frontend không tự publish MQTT và không cập nhật trực tiếp Database.

Các chức năng chính của Frontend trong phạm vi hệ thống gồm:

- Đăng nhập và duy trì phiên làm việc theo cơ chế xác thực của Backend.
- Hiển thị nhiệt độ, độ ẩm và ánh sáng mới nhất.
- Hiển thị lịch sử dữ liệu cảm biến theo phạm vi truy vấn được hỗ trợ.
- Hiển thị trạng thái Online/Offline của nút ESP8266.
- Hiển thị desired_state và reported_state của Fan và Light theo cách dễ phân biệt.
- Cho phép người dùng gửi yêu cầu bật/tắt Fan hoặc Light.
- Hiển thị trạng thái xử lý lệnh PENDING trong thời gian chờ ESP8266 xác nhận và SUCCESS sau khi hoàn tất.
- Hiển thị lịch sử thao tác điều khiển từ action_history.

Backend trả HTTP 202 và actionId khi nhận lệnh. Frontend giữ trạng thái chờ, gọi GET /api/state để đối soát và chỉ đổi công tắc khi có phản hồi SUCCESS từ ESP8266.

CHƯƠNG 3. THIẾT KẾ CƠ SỞ DỮ LIỆU



Hình 2 - Thiết kế DB

3.1 Bảng user

Bảng user được sử dụng để lưu trữ thông tin tài khoản của người dùng trong hệ thống. Thông tin trong bảng phục vụ chức năng đăng nhập, xác thực và xác định người thực hiện các thao tác điều khiển thiết bị.

STT	Tên trường	Kiểu SQLite	Mô tả
1	id	INTEGER	Khóa chính
2	username	TEXT UNIQUE	Tên đăng nhập
3	email	TEXT UNIQUE	Email
4	password_hash	TEXT	BCrypt hash
5	full_name	TEXT	Họ tên hiển thị
6	role	TEXT	USER hoặc ADMIN
7	created_at	TEXT	Thời điểm tạo UTC

Bảng 4 - Bảng DB User

3.2 Bảng iot_node

Bảng iot_node lưu thông tin của nút IoT trung tâm trong hệ thống, cụ thể là ESP8266. Bảng này giúp Backend quản lý thông tin nhận dạng của phần cứng và trạng thái kết nối Online/Offline.

STT	Tên trường	Kiểu SQLite	Mô tả
1	id	INTEGER	Khóa chính
2	name	TEXT	Tên nút
3	device_type	TEXT	ESP8266
4	mac_address	TEXT	MAC, có thể NULL
5	is_online	INTEGER	0 hoặc 1
6	last_seen	TEXT	Lần nhận bản tin cuối
7	created_at	TEXT	Thời điểm tạo UTC

Bảng 5 - Bảng DB iot-node

3.3 Bảng sensor

Bảng sensor lưu thông tin mô tả các cảm biến được kết nối với ESP8266. Đây là bảng lưu thông tin cố định của cảm biến, không lưu trực tiếp các giá trị đo theo thời gian. Trong hệ thống hiện tại, cảm biến thu thập ba nhóm dữ liệu gồm nhiệt độ, độ ẩm và ánh sáng.

STT	Tên trường	Kiểu SQLite	Mô tả
1	id	INTEGER	Khóa chính
2	iot_node_id	INTEGER FK	Tham chiếu iot_node
3	name	TEXT	Tên cảm biến
4	type	TEXT UNIQUE	TEMPERATURE/HUMIDITY/LIGHT
5	unit	TEXT	°C, %, ADC
6	mqtt_topic	TEXT UNIQUE	Topic nhận dữ liệu
7	is_active	INTEGER	0 hoặc 1
8	created_at	TEXT	Thời điểm tạo UTC

Bảng 6 - Bảng DB sensor

3.4 Bảng sensor_data

Bảng sensor_data được sử dụng để lưu các giá trị thực tế do cảm biến đo được theo thời gian. Khác với bảng sensor, bảng này có thể phát sinh rất nhiều bản ghi vì mỗi lần ESP8266 gửi dữ liệu cảm biến mới, Backend có thể tạo thêm một bản ghi trong sensor_data.

STT	Tên trường	Kiểu SQLite	Mô tả
1	id	INTEGER	Khóa chính
2	sensor_id	INTEGER FK	Tham chiếu sensor
3	value	REAL	Giá trị đo
4	recorded_at	TEXT	Thời điểm ghi UTC

Bảng 7 - Bảng DB sensor data

3.5 Bảng actuator

Bảng actuator lưu thông tin các thiết bị được ESP8266 điều khiển. Trong mô hình hiện tại, hệ thống sử dụng LED để mô phỏng hai actuator là Fan và Light. Điểm quan trọng là hệ thống lưu riêng desired_state (trạng thái người dùng mong muốn) và reported_state (trạng thái thực tế được ESP8266 xác nhận).

STT	Tên trường	Kiểu SQLite	Mô tả
1	id	INTEGER	Khóa chính
2	iot_node_id	INTEGER FK	Tham chiếu iot_node
3	name	TEXT	Đèn LED hoặc Quạt
4	type	TEXT UNIQUE	LIGHT hoặc FAN
5	desired_state	TEXT	ON/OFF mong muốn
6	reported_state	TEXT	ON/OFF đã xác nhận
7	mqtt_control_topic	TEXT UNIQUE	Topic gửi lệnh
8	mqtt_status_topic	TEXT UNIQUE	Topic phản hồi
9	updated_at	TEXT	Lần cập nhật UTC

Bảng 8 - Bảng DB actuator

3.6 Bảng action_history

Bảng action_history được sử dụng để lưu lịch sử các thao tác điều khiển Fan và Light của người dùng. Mỗi lần người dùng gửi lệnh bật hoặc tắt thiết bị, Backend tạo một bản ghi lịch sử để theo dõi quá trình thực hiện lệnh.

STT	Tên trường	Kiểu SQLite	Mô tả
1	id	INTEGER	Khóa chính
2	user_id	INTEGER FK	Người gửi lệnh
3	actuator_id	INTEGER FK	Thiết bị nhận lệnh
4	action	TEXT	ON hoặc OFF
5	status	TEXT	PENDING/SUCCESS/FAILED
6	created_at	TEXT	Thời điểm kích hoạt UTC
7	response_time_ms	INTEGER	Thời gian phản hồi ms
8	result	TEXT	Diễn giải kết quả

Bảng 9 - Bảng DB action history

3.7 Quan hệ giữa các bảng

- iot_node (1) -> (N) sensor: Một ESP8266 có thể quản lý nhiều cảm biến.
- iot_node (1) -> (N) actuator: Một ESP8266 có thể điều khiển nhiều thiết bị chấp hành.
- sensor (1) -> (N) sensor_data: Một cảm biến có thể tạo ra nhiều bản ghi dữ liệu theo thời gian.
- user (1) -> (N) action_history: Một người dùng có thể thực hiện nhiều thao tác điều khiển.
- actuator (1) -> (N) action_history: Một thiết bị chấp hành có thể có nhiều bản ghi lịch sử điều khiển.

CHƯƠNG 4. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

4.1 Yêu cầu hệ thống

4.1.1 *Yêu cầu chức năng*

- Đăng nhập và xác thực người dùng.
- Xem dữ liệu nhiệt độ, độ ẩm và ánh sáng mới nhất.
- Xem lịch sử dữ liệu cảm biến theo loại cảm biến và khoảng thời gian.
- Bật/tắt Fan và Light thông qua giao diện web.
- Xem trạng thái kết nối của ESP8266 và lịch sử điều khiển thiết bị.

4.1.2 *Yêu cầu phi chức năng*

- Tính nhất quán: thuật ngữ và trạng thái phải thống nhất giữa API, Database, MQTT và giao diện.
- Khả năng truy vết: mỗi lệnh điều khiển phải có bản ghi lịch sử và trạng thái xử lý.
- Khả năng phục hồi: trạng thái mong muốn không bị mất khi Backend khởi động lại hoặc ESP8266 kết nối lại.
- Tính dễ sử dụng: giao diện phân biệt rõ trạng thái đã xác nhận, đang đồng bộ và thiết bị offline.
- Bảo mật cơ bản: API cần xác thực người dùng; mật khẩu không được lưu ở dạng văn bản thuần trong triển khai thực tế.

4.2 Mô hình Use Case và Sequence Diagram

Phần này xác định các chức năng mà người dùng tương tác trực tiếp. Mỗi Use Case được mô tả theo mục tiêu, actor, tiền điều kiện, luồng chính, luồng ngoại lệ và hậu điều kiện để bảo đảm có thể chuyển tiếp sang thiết kế API và kiểm thử.

Mã Use Case	Tên Use Case	Actor chính
UC01	Đăng nhập hệ thống	User
UC02	Xem thông số môi trường	User
UC03	Xem lịch sử cảm biến	User
UC04	Bật/tắt hai LED	User
UC05	Xem lịch sử điều khiển	User

Bảng 10 - Danh sách UC

4.2.1 *UC01 - Đăng nhập hệ thống*

Mục tiêu: Cho phép người dùng xác thực để truy cập vào ứng dụng web và sử dụng các chức năng của hệ thống.

Actor: User

Tiền điều kiện:

- Người dùng đã có tài khoản hợp lệ trong hệ thống.
- Ứng dụng web và backend đang hoạt động.

Luồng chính:

1. Người dùng truy cập trang đăng nhập.
2. Người dùng nhập tên đăng nhập hoặc email và mật khẩu.
3. Frontend gửi thông tin đăng nhập đến Backend thông qua API.
4. Backend kiểm tra thông tin tài khoản trong cơ sở dữ liệu.
5. Nếu thông tin hợp lệ, Backend xác thực người dùng và trả về kết quả đăng nhập thành công.
6. Frontend chuyển người dùng đến trang Dashboard.

Luồng ngoại lệ / thay thế:

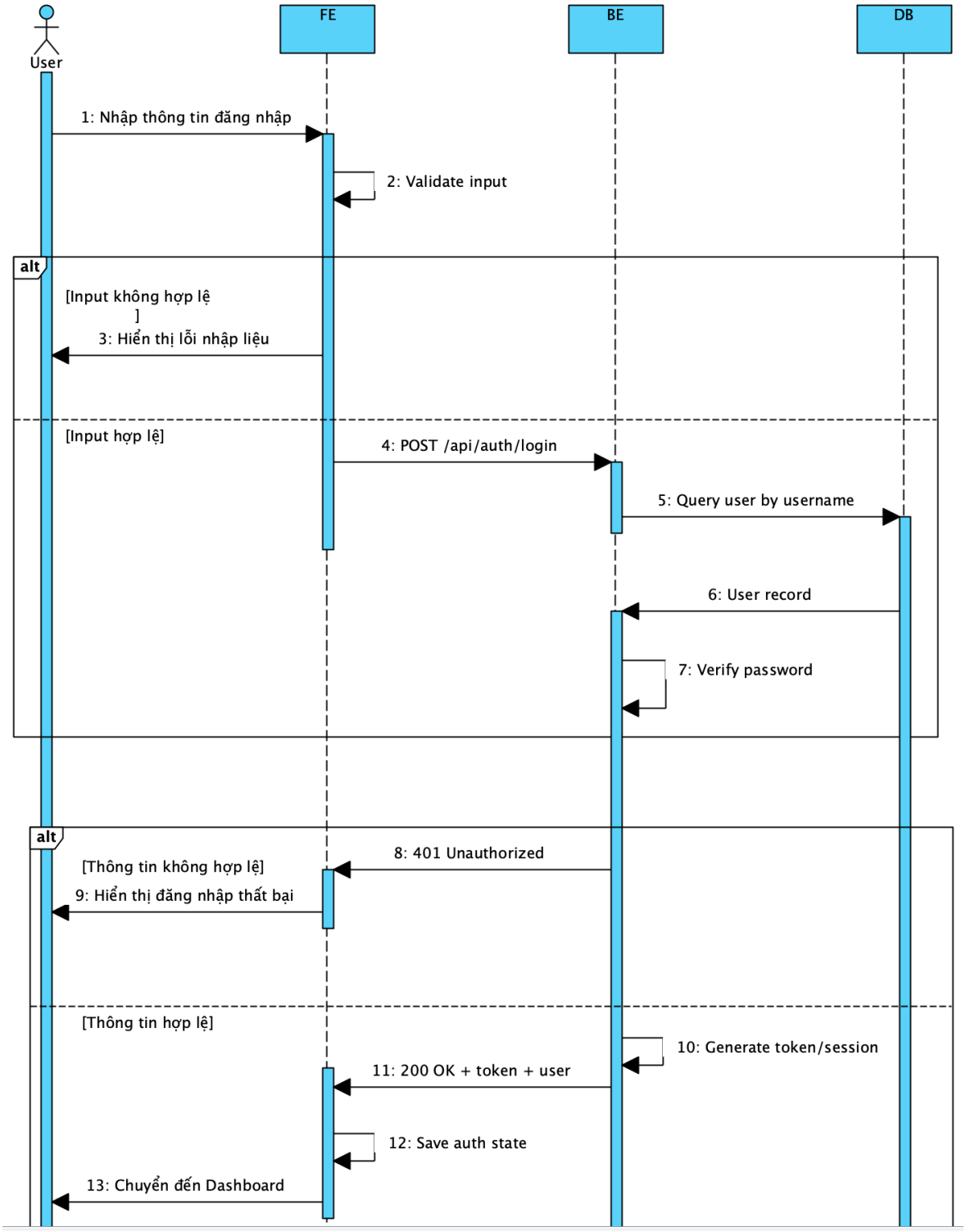
Nếu tên đăng nhập/email hoặc mật khẩu không chính xác, hệ thống hiển thị thông báo đăng nhập thất bại.

Nếu Backend hoặc cơ sở dữ liệu không phản hồi, hệ thống thông báo lỗi kết nối.

Hậu điều kiện:

Người dùng được xác thực và có thể truy cập các chức năng của hệ thống.

4.2.1.1 Mô tả Sequence Diagram



Hình 3 - Sequence Diagram UC1

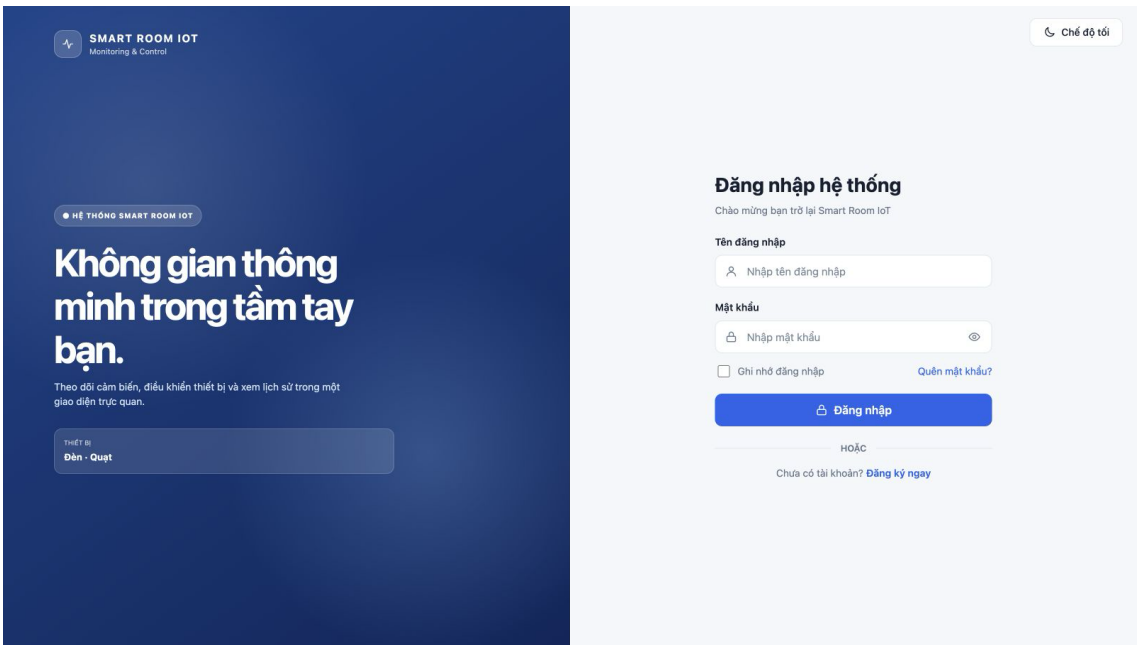
1. Người dùng nhập tên đăng nhập hoặc email cùng mật khẩu. Frontend gửi POST /api/auth/login.

2. AuthService tìm tài khoản trong bảng user và kiểm tra mật khẩu bằng BCrypt. Backend tạo HttpSession, đổi session ID và sinh CSRF token.

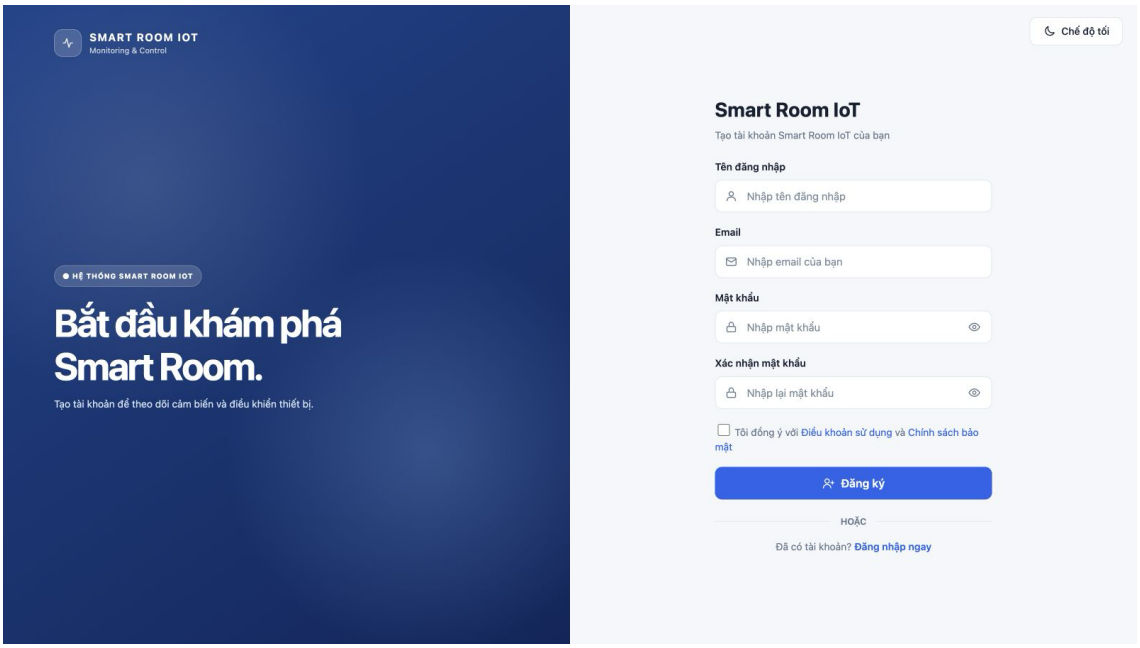
3. Frontend giữ token theo tùy chọn Ghi nhớ đăng nhập rồi chuyển sang Dashboard. Khi sai thông tin, API trả 401.

4. Người chưa có tài khoản dùng màn hình Đăng ký; Backend tạo user mới qua POST /api/auth/register.

4.2.2 *Màn hình Đăng nhập*



Hình 4 - Màn hình Đăng nhập



Hình 5 - Màn hình Đăng ký tài khoản

4.2.3 *UC02 - Xem thông số môi trường hiện tại*

Mục tiêu: Cho phép người dùng xem giá trị mới nhất của nhiệt độ, độ ẩm và cường độ ánh sáng trong phòng.

Actor: User

Tiền điều kiện:

Người dùng đã đăng nhập vào hệ thống.

Hệ thống đã nhận được dữ liệu cảm biến từ ESP8266.

Luồng chính:

1. Người dùng mở trang Dashboard.

2. Frontend gửi yêu cầu lấy dữ liệu cảm biến mới nhất đến Backend.

3. Backend truy vấn dữ liệu mới nhất của các cảm biến trong cơ sở dữ liệu.

4. Backend trả về các giá trị nhiệt độ, độ ẩm và cường độ ánh sáng.

5. Frontend hiển thị các thông số lên Dashboard cho người dùng.

Luồng ngoại lệ / thay thế:

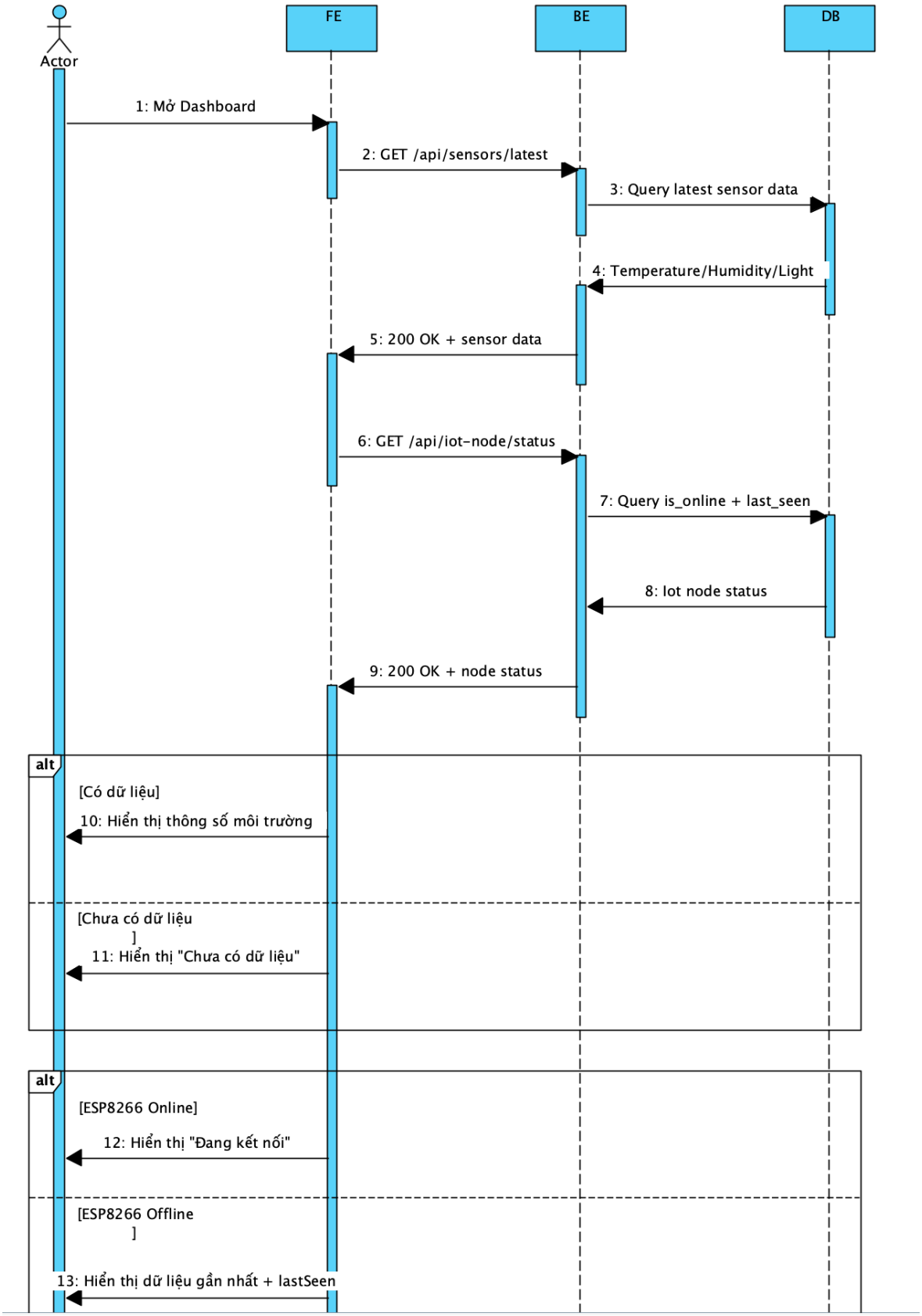
Nếu chưa có dữ liệu cảm biến, hệ thống hiển thị trạng thái chưa có dữ liệu.

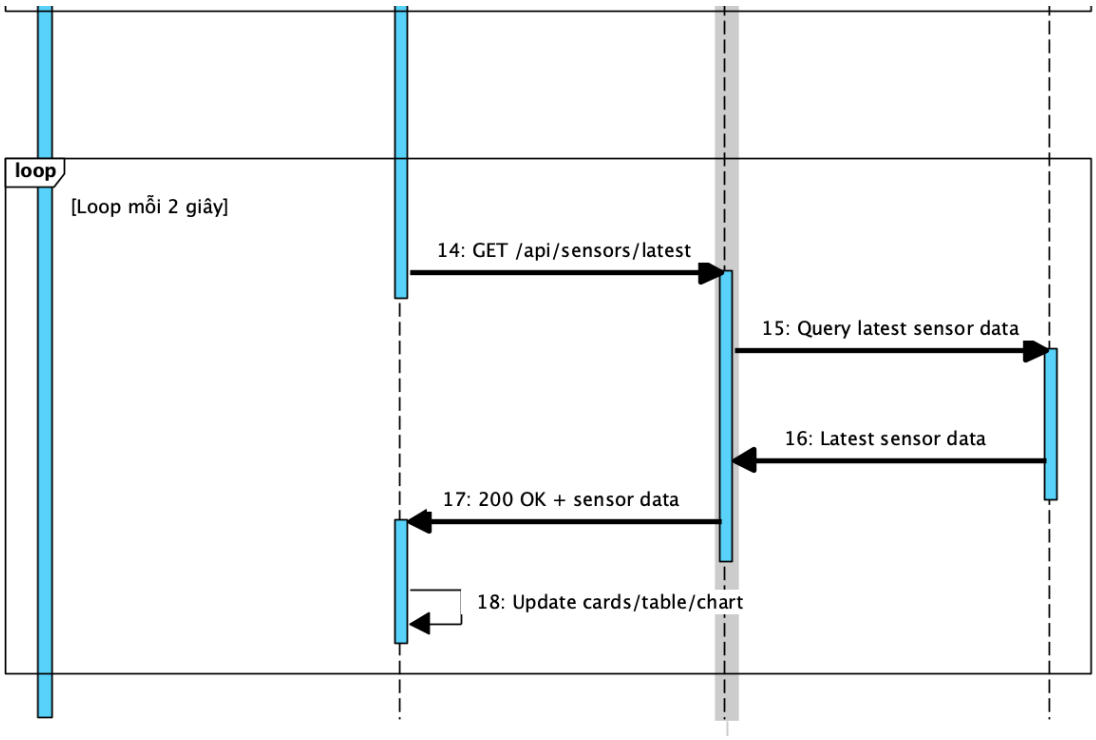
Nếu ESP8266 đang offline, hệ thống vẫn hiển thị dữ liệu gần nhất đã lưu và có thể kèm trạng thái thiết bị offline.

Hậu điều kiện:

Người dùng xem được các thông số môi trường mới nhất mà hệ thống đang lưu trữ.

4.2.3.1 Mô tả Sequence Diagram


Actor
FE
BE
DB
1: Mở Dashboard
2: GET /api/sensors/latest
3: Query latest sensor data
4: Temperature/Humidity/Light
5: 200 OK + sensor data
6: GET /api/iot-node/status
7: Query is_online + last_seen
8: Iot node status
9: 200 OK + node status
alt
[Có dữ liệu]
10: Hiển thị thông số môi trường
[Chưa có dữ liệu
]
11: Hiển thị "Chưa có dữ liệu"
alt
[ESP8266 Online]
12: Hiển thị "Đang kết nối"
[ESP8266 Offline
]
13: Hiển thị dữ liệu gần nhất + lastSeen



Hình 6 - Sequence Diagram UC2

1. Dashboard hiển thị ba thẻ cảm biến, hai công tắc LED, biểu đồ và ba lệnh điều khiển gần nhất.

2. Frontend lấy toàn bộ trạng thái từ GET /api/state để các khu vực trên Dashboard dùng cùng một nguồn dữ liệu.

3. Khi mở Dashboard, Frontend gọi GET /api/state với cookie phiên hiện tại.

4. Backend đọc sensor_data cùng sensor, action_history cùng actuator và trạng thái thiết bị từ SQLite.

5. Mỗi bản ghi cảm biến gồm type, value, unit và recordedAt. Đơn vị ánh sáng là ADC.

6. Backend trả JSON gồm sensors, actions, devices và online; Frontend chọn mẫu mới nhất theo từng loại.

7. Trạng thái online được suy ra từ last_seen của ESP8266 trong 12 giây gần nhất.

8. Backend không lưu địa chỉ IP của ESP8266 để hiển thị trên Dashboard.

9. Frontend hiển thị nhiệt độ °C, độ ẩm % và ánh sáng ADC; biểu đồ lấy các mẫu đã lưu.

10. Nếu mạch ngừng gửi bản tin, dữ liệu cũ vẫn được hiển thị nhưng chỉ báo online chuyển sang ngoại tuyến sau ngưỡng 12 giây.

11. Trong khi Dashboard mở, Frontend gọi lại GET /api/state mỗi 3 giây.

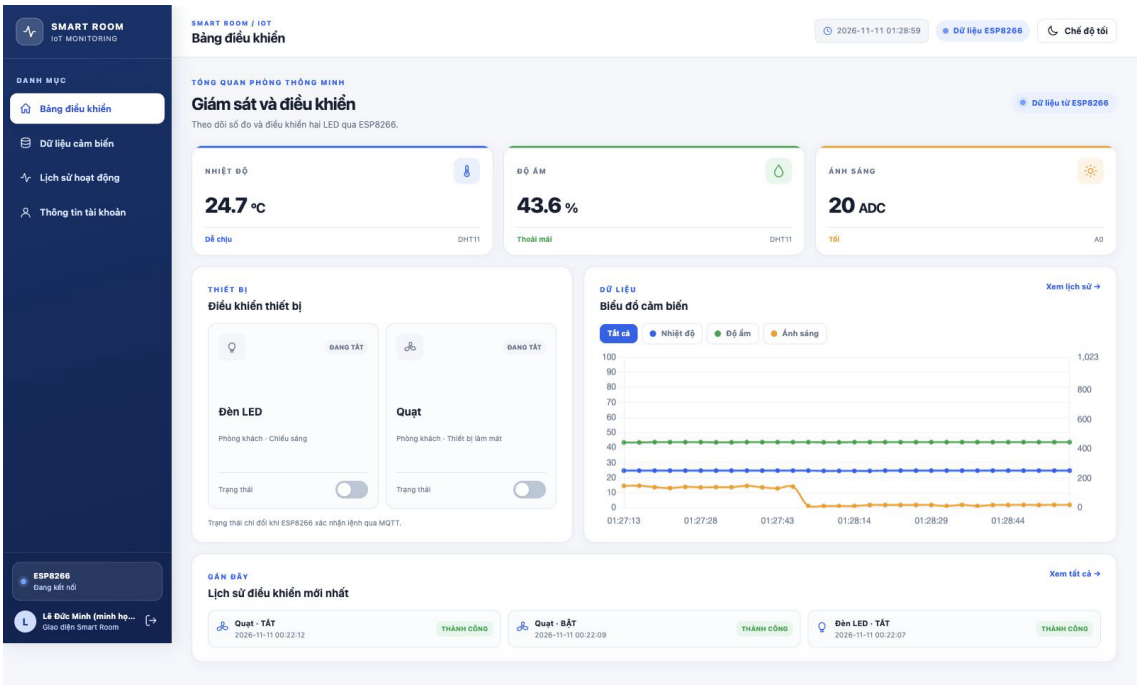
12. Frontend vẽ lại thẻ, công tắc, lịch sử ngắn và biểu đồ từ dữ liệu mới; khi rời trang, bộ hẹn giờ của trang kết thúc.

13. Khi chưa có mẫu, giao diện hiển thị “Chưa có dữ liệu” thay cho một giá trị đo giả.

14. Khi mạch offline, Frontend tiếp tục đọc dữ liệu đã lưu và báo trạng thái kết nối theo phản hồi backend.

15. Nếu phiên hết hạn, API trả 401 và Frontend chuyển về trang đăng nhập.

4.2.4 *Màn hình Dashboard*



Hình 7 - Màn hình Dashboard

4.2.5 *UC03 - Xem lịch sử dữ liệu cảm biến*

Mục tiêu:

Cho phép người dùng tra cứu và theo dõi dữ liệu lịch sử của các cảm biến theo loại dữ liệu hoặc theo thời gian.

Actor: User

Tiền điều kiện:

- Người dùng đã đăng nhập vào hệ thống.
- Cơ sở dữ liệu đã có dữ liệu lịch sử từ các cảm biến.
- Backend và Database đang hoạt động.

Luồng chính

1. Người dùng mở trang Lịch sử cảm biến.
2. Frontend hiển thị một ô tìm kiếm, dropdown loại tìm kiếm gồm Tất cả, Nhiệt độ, Độ ẩm, Ánh sáng và Thời gian; đồng thời hiển thị khu vực bảng và biểu đồ.
3. Người dùng chọn loại tìm kiếm, nhập từ khóa và nhấn Tìm kiếm.
4. Frontend tải GET /api/state, rồi lọc theo loại cảm biến, giá trị hoặc thời gian ngay trên trình duyệt.
5. Backend kiểm tra tham số và JOIN bảng sensor_data với sensor theo điều kiện sensor_data.sensor_id = sensor.id.

6. Backend lọc dữ liệu theo searchType và keyword, sau đó nhận các bản ghi phù hợp từ Database.
7. Backend trả về các trường id, type, value, unit và recordedAt.
8. Frontend hiển thị kết quả trên bảng và cập nhật biểu đồ tương ứng.

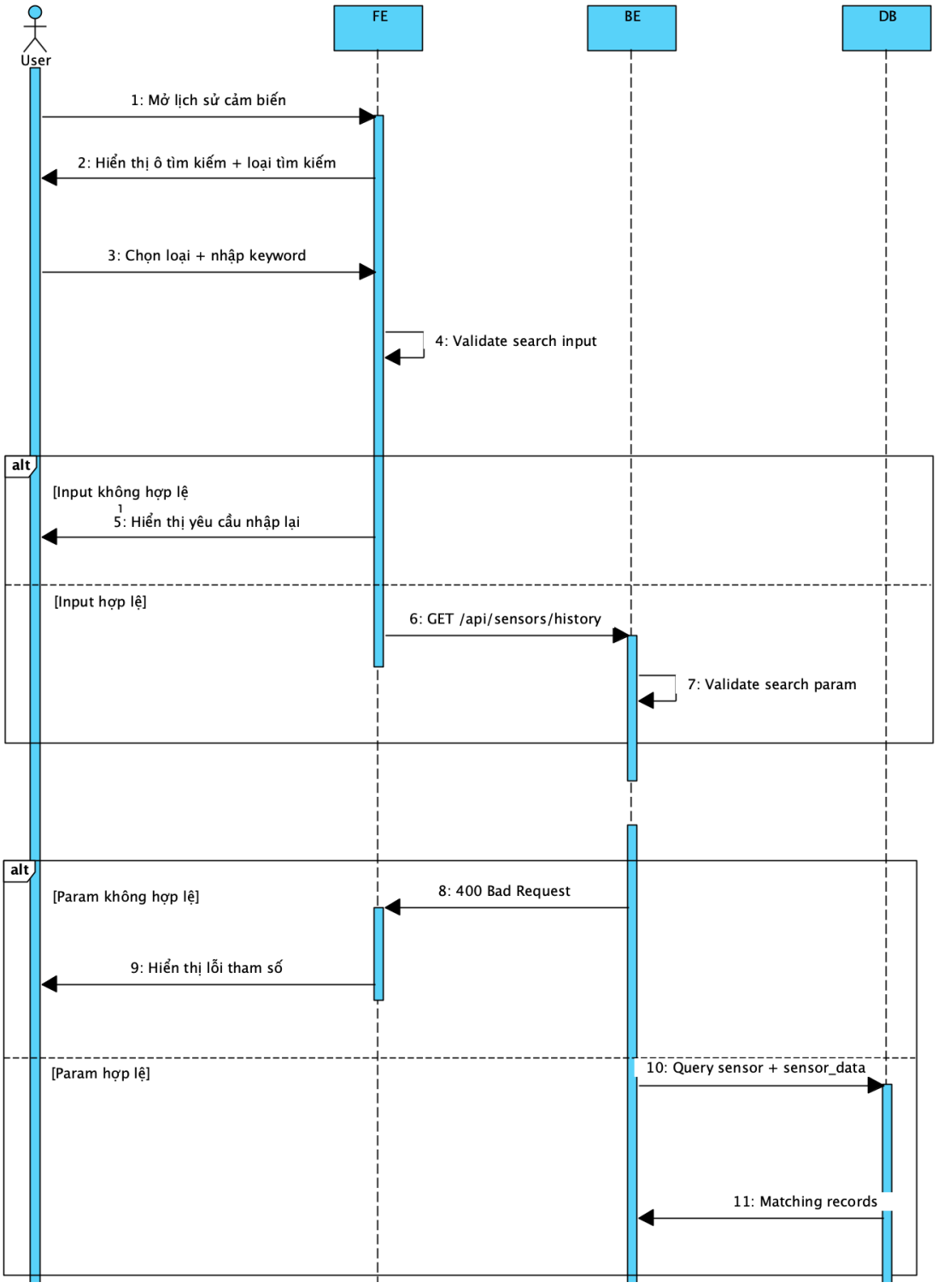
Luồng ngoại lệ / thay thế

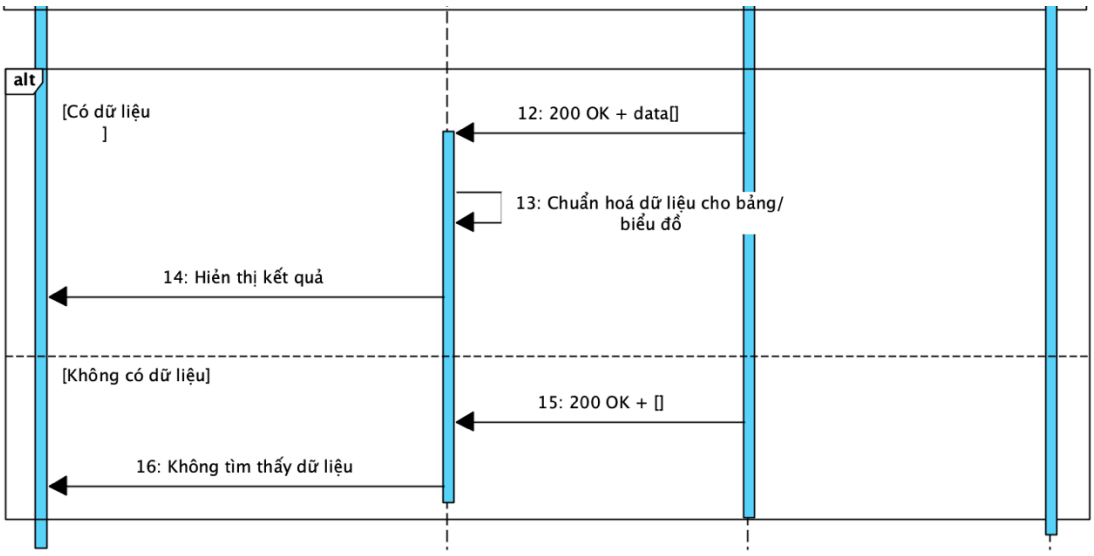
- Nếu dữ liệu nhập không hợp lệ, Frontend yêu cầu người dùng nhập lại và không gửi yêu cầu.
- Frontend kiểm tra từ khóa thời gian trước khi lọc. API GET /api/sensors/history vẫn có thể dùng độc lập trong Postman.
- Nếu không có bản ghi phù hợp, Backend trả danh sách rỗng; Frontend hiển thị thông báo “Không tìm thấy dữ liệu phù hợp”.
- Khi người dùng nhấn Đặt lại, Frontend xóa từ khóa, đưa loại tìm kiếm về Tất cả và tải lại dữ liệu mặc định.

Hậu điều kiện:

- Kết quả tra cứu được hiển thị đúng theo điều kiện tìm kiếm
- Hệ thống không làm thay đổi dữ liệu lịch sử đã lưu.

4.2.5.1 Mô tả Sequence Diagram

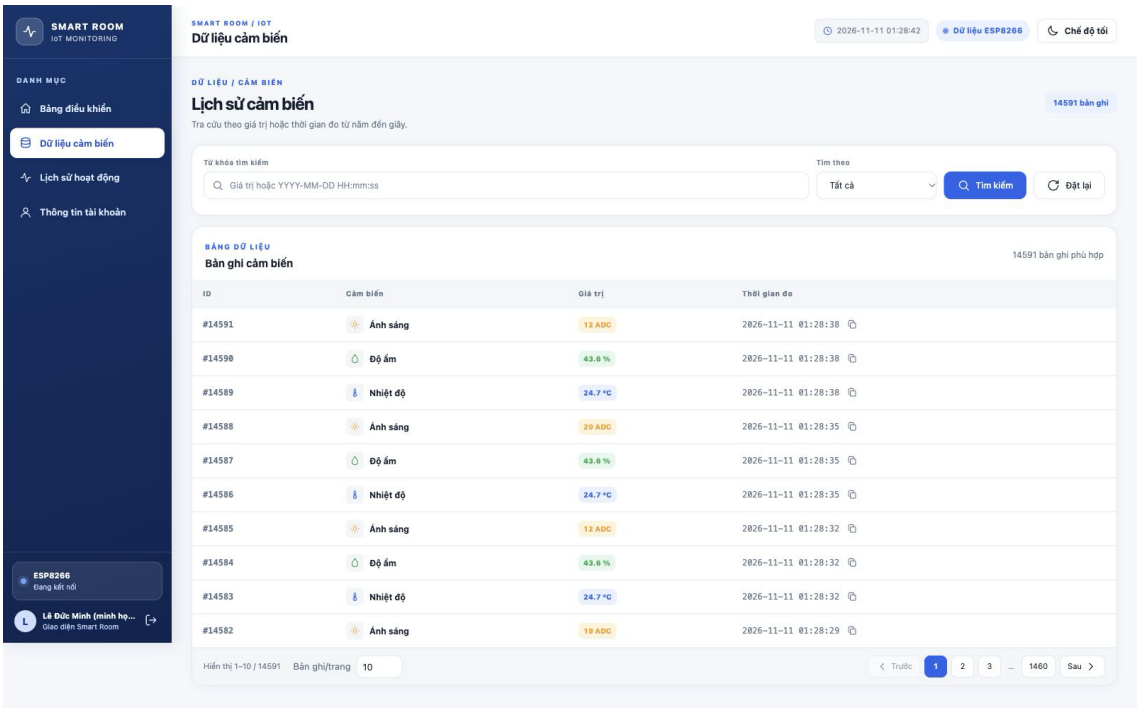

User
FE
BE
DB
1: Mở lịch sử cảm biến
2: Hiển thị ô tìm kiếm + loại tìm kiếm
3: Chọn loại + nhập keyword
4: Validate search input
alt
[Input không hợp lệ
5: Hiển thị yêu cầu nhập lại
[Input hợp lệ]
6: GET /api/sensors/history
7: Validate search param
alt
[Param không hợp lệ]
8: 400 Bad Request
9: Hiển thị lỗi tham số
[Param hợp lệ]
10: Query sensor + sensor_data
11: Matching records



Hình 8 - Sequence Diagram UC3

1. Trang Lịch sử cảm biến gọi GET /api/state mỗi 5 giây. Backend đọc các bản ghi sensor_data đã lưu.
2. Frontend lọc theo loại, số đo hoặc thời gian, hiển thị biểu đồ và phân trang 10 dòng. Nút sao chép lấy chuỗi thời gian đang hiển thị.
3. Endpoint GET /api/sensors/history hỗ trợ tìm kiếm và phân trang ở backend khi thử API độc lập.

4.2.6 *Màn hình Lịch sử cảm biến*



Hình 9 - Màn hình Lịch sử cảm biến

4.2.7 *UC04 – Điều khiển bật/tắt thiết bị*

Actor: User.

Mục tiêu:

Cho phép người dùng điều khiển trạng thái bật/tắt của các thiết bị trong phòng từ ứng dụng web, đồng thời bảo đảm trạng thái mong muốn và trạng thái thực tế được lưu trữ, đồng bộ với ESP8266.

Tiền điều kiện:

- Người dùng đã đăng nhập.
- Thiết bị cần điều khiển đã được khai báo trong hệ thống.
- Backend và MQTT Broker đang hoạt động.

Luồng chính:

1. Người dùng mở Dashboard và xem trạng thái hiện tại của thiết bị.
2. Người dùng chọn thiết bị cần điều khiển, ví dụ Fan hoặc Light.
3. Người dùng chọn trạng thái ON hoặc OFF.
4. Frontend gửi yêu cầu thay đổi trạng thái thiết bị đến Backend.
5. Backend cập nhật desired_state của thiết bị trong cơ sở dữ liệu.
6. Backend tạo bản ghi trong action_history với trạng thái PENDING.
7. Backend publish lệnh điều khiển lên MQTT Broker thông qua topic tương ứng với thiết bị.
8. ESP8266 nhận lệnh MQTT và thay đổi GPIO tương ứng để bật hoặc tắt LED mô phỏng.
9. ESP8266 publish trạng thái thực tế của thiết bị về MQTT Broker.
10. Backend nhận trạng thái phản hồi và cập nhật reported_state.
11. Backend cập nhật trạng thái bản ghi action_history từ PENDING sang SUCCESS.
12. Frontend lấy và hiển thị trạng thái mới nhất của thiết bị.

Luồng ngoại lệ / thay thế:

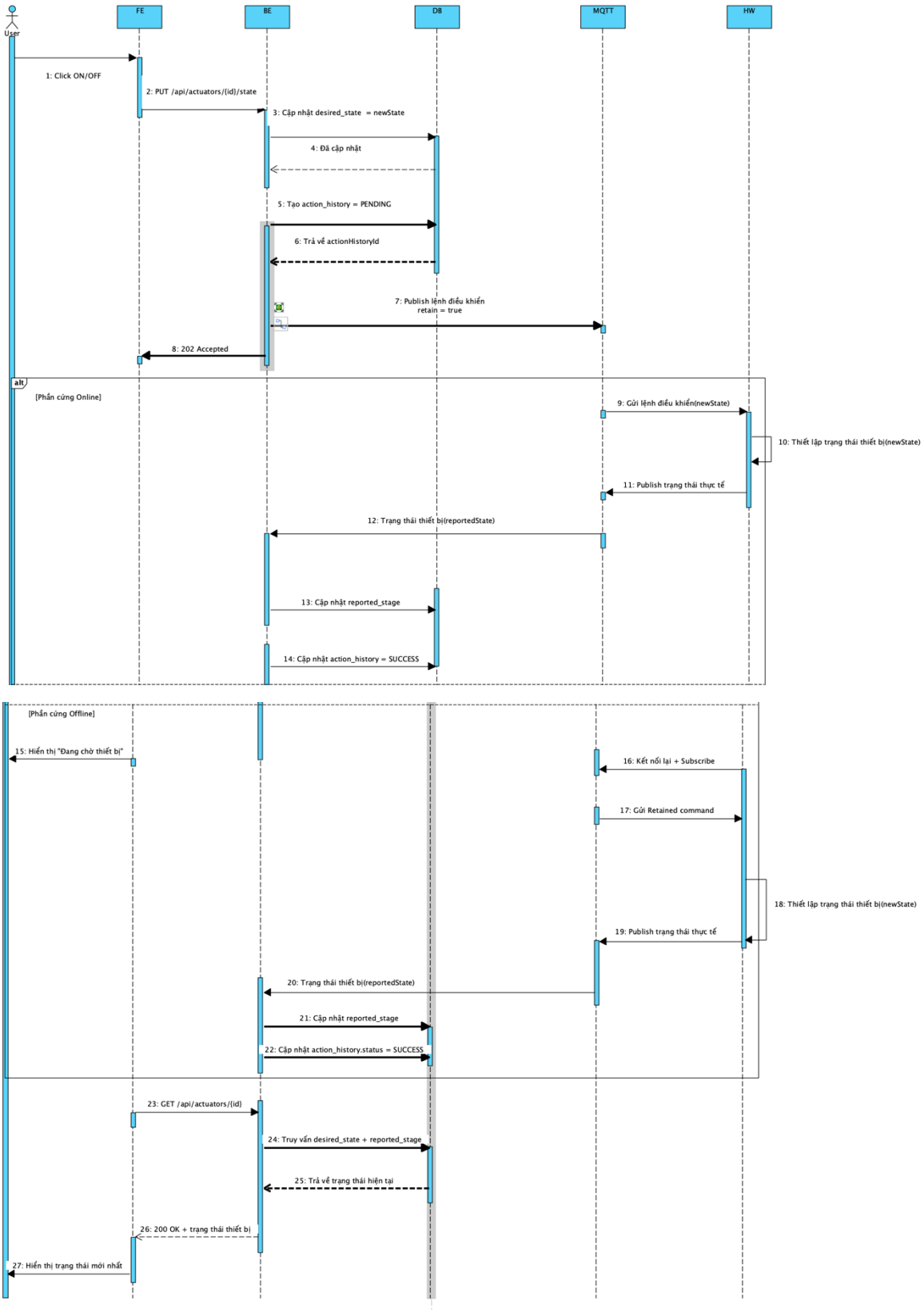
1. Nếu ESP8266 mất kết nối, broker giữ lệnh điều khiển cuối cùng bằng retained message. Khi mạch kết nối lại, ESP8266 thực hiện lệnh và publish trạng thái.
2. Nếu sau 10 giây chưa có xác nhận, Backend đổi lệnh sang FAILED. Nếu phản hồi đúng đến muộn, Backend giữ bản ghi FAILED và thêm một bản ghi SUCCESS mới.
3. Nếu thiết bị không tồn tại, Backend trả lỗi 404 Not Found.
4. Nếu trạng thái yêu cầu không hợp lệ, Backend trả lỗi 400 Bad Request.

Hậu điều kiện:

- desired_state của thiết bị được lưu trong Database.

- Thao tác điều khiển được ghi vào action_history.
- reported_state được cập nhật sau khi ESP8266 xác nhận.
- Trạng thái thiết bị không bị mất khi reload trang hoặc Backend restart.
- Khi ESP8266 reconnect, thiết bị có thể đồng bộ lại trạng thái mong muốn.

4.2.7.2 Mô tả Sequence Diagram



Hình 10 - Sequence Diagram UC4

Người dùng đổi công tắc. Frontend gửi POST /api/devices/LIGHT hoặc /api/devices/FAN với state ON/OFF và CSRF token.

RoomService lưu desired_state, tạo action_history ở trạng thái PENDING rồi publish MQTT retained. API trả HTTP 202 với actionId.

ESP8266 đổi chân D7 hoặc D6 và publish trạng thái thực tế. TelemetryService cập nhật reported_state và chuyển lệnh sang SUCCESS.

Nếu 10 giây không có phản hồi, Backend ghi FAILED. Xác nhận đến sau khi mạch kết nối lại được ghi thành dòng SUCCESS riêng; giao diện đọc trạng thái từ GET /api/state.

4.2.8 *UC05 - Xem lịch sử điều khiển thiết bị*

Mục tiêu: Cho phép người dùng xem lại các thao tác bật/tắt quạt và đèn đã được thực hiện trong hệ thống.

Actor: User

Tiền điều kiện:

Người dùng đã đăng nhập.

Hệ thống đã có dữ liệu lịch sử điều khiển.

Luồng chính:

1. Người dùng mở trang Action History.

2. Frontend gửi yêu cầu lấy lịch sử điều khiển đến Backend.

3. Backend truy vấn bảng Action History trong cơ sở dữ liệu.

4. Backend trả danh sách các thao tác về Frontend.

5. Frontend hiển thị thời gian, thiết bị, hành động, người thực hiện và trạng thái xử lý của từng thao tác.

Luồng ngoại lệ / thay thế:

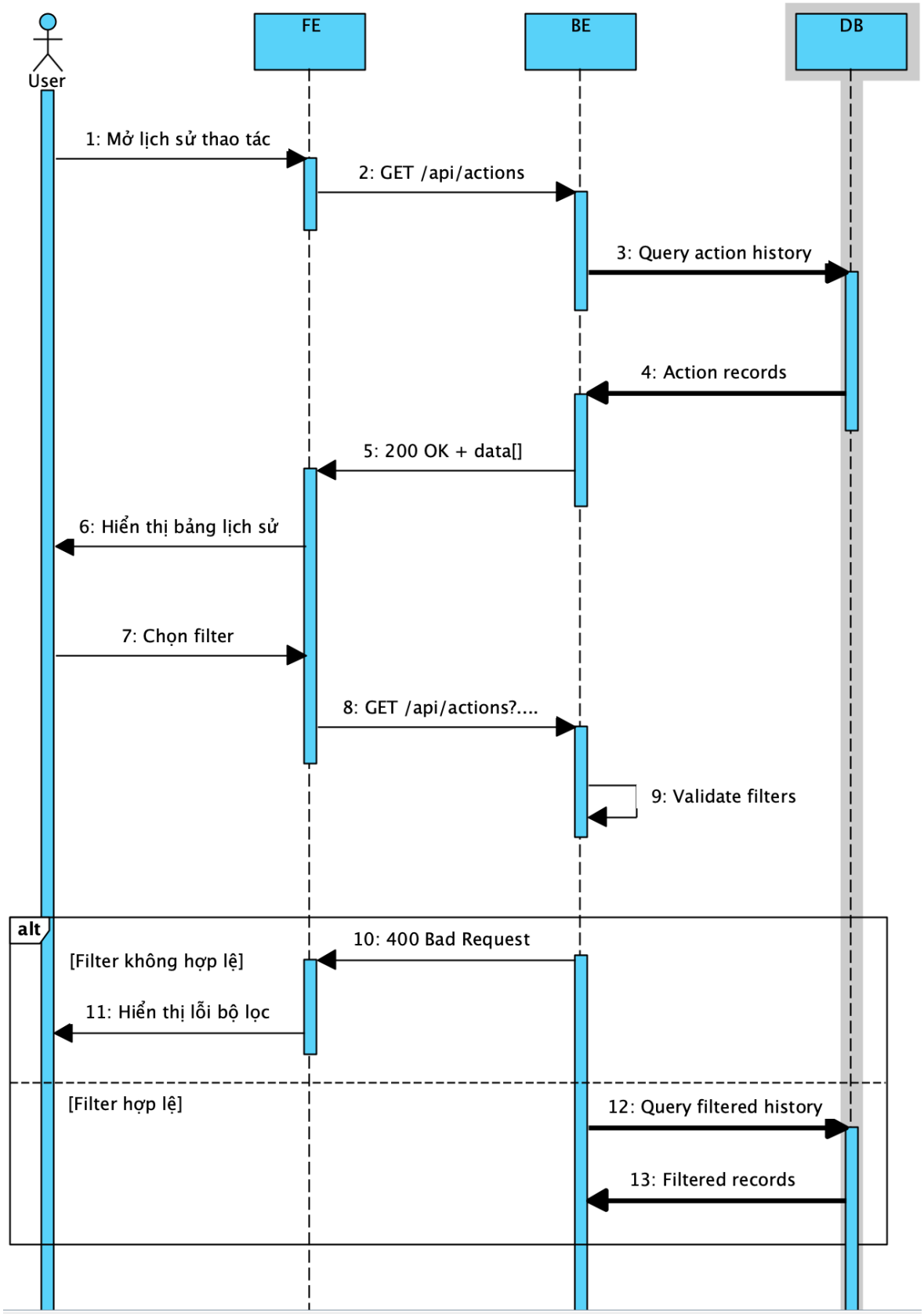
Nếu chưa có thao tác điều khiển nào, hệ thống hiển thị danh sách trống hoặc thông báo chưa có dữ liệu.

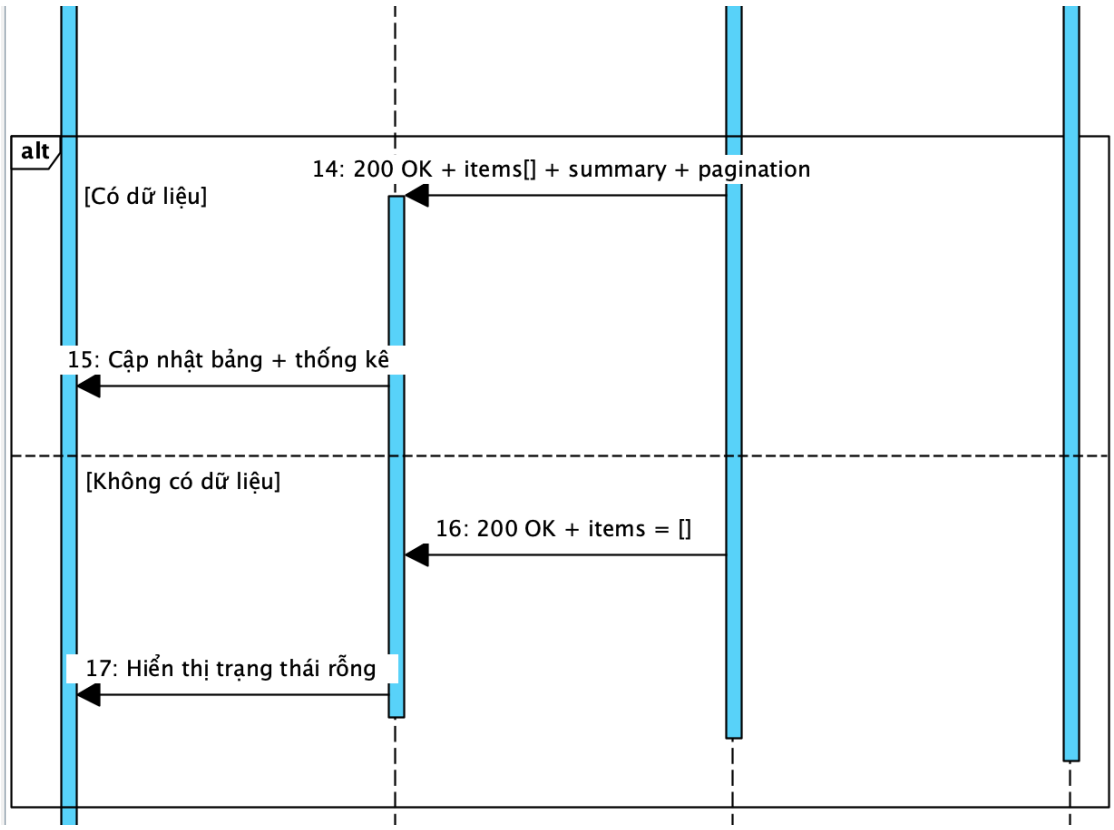
Nếu truy vấn cơ sở dữ liệu thất bại, hệ thống hiển thị thông báo lỗi.

Hậu điều kiện:

Người dùng xem được lịch sử các thao tác điều khiển thiết bị đã lưu trong hệ thống.

4.2.8.1 Mô tả Sequence Diagram


User
FE
BE
DB
1: Mở lịch sử thao tác
2: GET /api/actions
3: Query action history
4: Action records
5: 200 OK + data[]
6: Hiển thị bảng lịch sử
7: Chọn filter
8: GET /api/actions?....
9: Validate filters
alt
[Filter không hợp lệ]
10: 400 Bad Request
11: Hiển thị lỗi bộ lọc
[Filter hợp lệ]
12: Query filtered history
13: Filtered records



Hình 11 - Sequence Diagram UC5

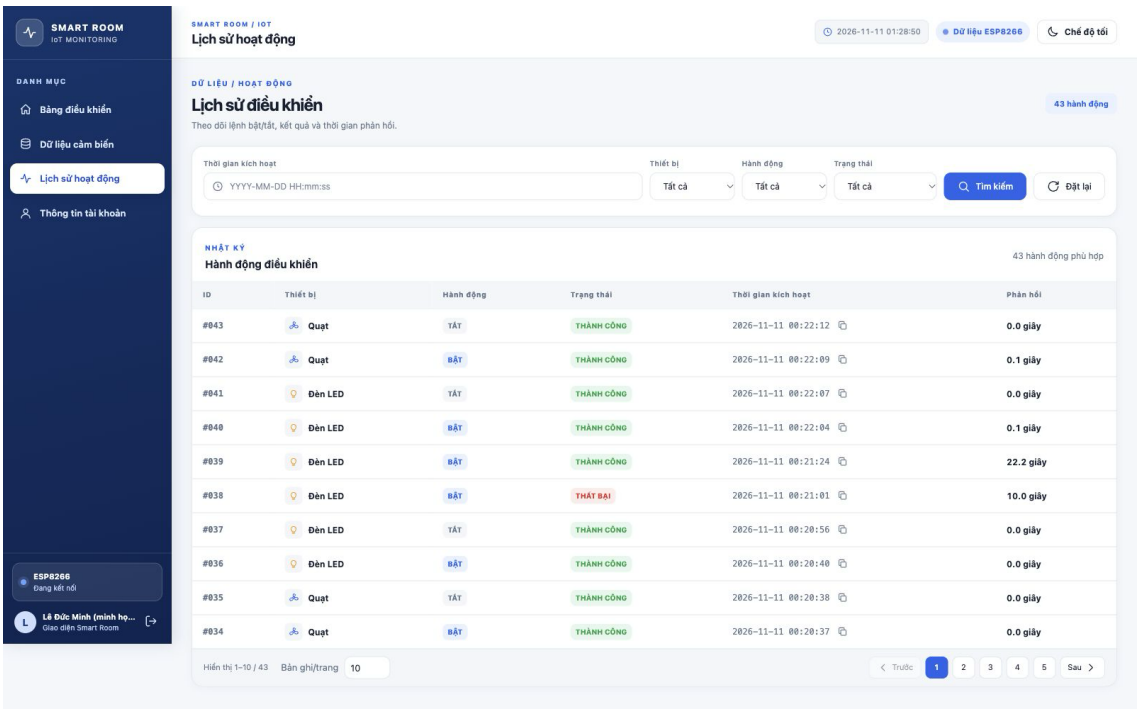
1. Trang Lịch sử hoạt động tải GET /api/state mỗi 5 giây và lấy mảng actions.

2. Frontend lọc theo thiết bị, hành động, trạng thái và thời gian rồi phân trang 10 dòng. Repository sắp xếp bản ghi mới nhất lên đầu.

3. Bảng hiển thị thời gian phản hồi của lệnh đã hoàn thành; lệnh PENDING hiện "Đang xử lý".

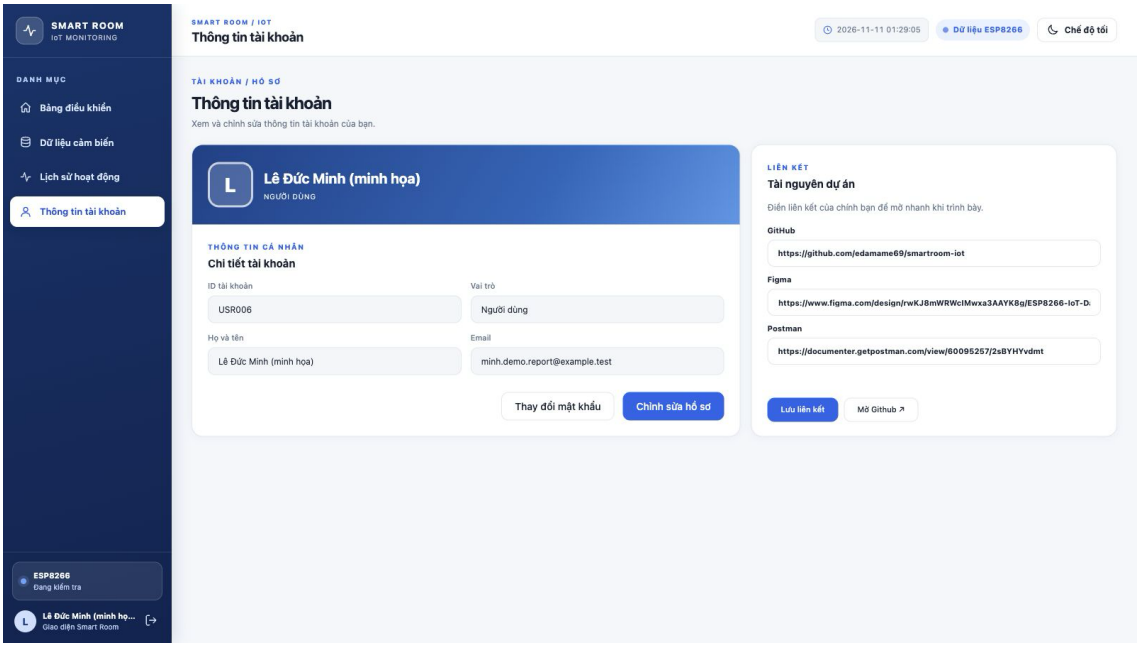
4. API GET /api/actions cung cấp cùng dữ liệu cho Postman với bộ lọc và phân trang phía backend.

4.2.9 *Màn hình lịch sử điều khiển thiết bị*



Hình 12 - Màn hình Lịch sử hoạt động

4.2.10 *Màn hình Profile*



Hình 13 - Màn hình Profile

CHƯƠNG 5. CÀI ĐẶT BACKEND VÀ CƠ SỞ DỮ LIỆU

5.1 Tổ chức mã nguồn

Backend sử dụng Spring Boot 3.5.6 và Java 17. ApiController nhận yêu cầu HTTP; AuthService xử lý tài khoản; RoomService thực hiện lệnh; TelemetryService nhận dữ liệu MQTT. SmartRoomRepository dùng JdbcTemplate để truy vấn SQLite. MqttGateway duy trì kết nối broker ở cổng 1884.

5.2 Xử lý lệnh bật tắt

ApiController kiểm tra phiên và CSRF token trước khi chuyển yêu cầu tới RoomService. Hai đường dẫn điều khiển hiện có là POST /api/devices/{type} và PUT /api/actuators/{id}/state. Cả hai trả HTTP 202 và actionId khi Backend tiếp nhận lệnh.

```
@PutMapping("/actuators/{id}/state")
ResponseEntity<Map<String, Object>> actuatorCommand(@PathVariable int id, @RequestBody Command input,
        HttpServletRequest request) {
    csrf(request);
    ApiException.require(input != null, HttpStatus.BAD_REQUEST, message: "Trạng thái không hợp lệ");
    String type = (String) room.actuator(id).get(key: "type");
    return ResponseEntity.status(HttpStatus.ACCEPTED).body(room.command(user(request), type, input.state()));
}
```

Hình 14 - ApiController nhận yêu cầu điều khiển và trả HTTP 202

RoomService kiểm tra LIGHT/FAN và ON/OFF, từ chối lệnh mới nếu thiết bị còn một lệnh PENDING, lưu desired_state và tạo action_history trước khi publish MQTT. Nếu broker không nhận lệnh, service đánh dấu FAILED và trả lỗi 503.

```
public Map<String, Object> command(int userId, String type, String state) {
    ApiException.require(type.equals(anObject: "FAN") || type.equals(anObject: "LIGHT"), HttpStatus.BAD_REQUEST, message
    ApiException.require(state != null && (state.equals(anObject: "ON") || state.equals(anObject: "OFF")),
            HttpStatus.BAD_REQUEST, message: "Trạng thái không hợp lệ");
    ApiException.require(!repository.hasPending(type), HttpStatus.CONFLICT, message: "Thiết bị đang xử lý lệnh khác");
    int actuatorId = repository.actuatorId(type);
    String now = Instant.now().toString();
    repository.setDesired(actuatorId, state, now);
    int actionId = repository.createAction(userId, actuatorId, state, now);
    try { mqtt.publish(type, state); }
    catch (Exception e) {
        repository.failPublish(actionId);
        throw new ApiException(HttpStatus.SERVICE_UNAVAILABLE, message: "Không kết nối được MQTT broker");
    }
    return Map.of(k1: "actionId", actionId, k2: "status", v2: "PENDING");
}
```

Hình 15 - RoomService lưu lệnh PENDING rồi gửi MQTT

TelemetryService nhận topic trạng thái do ESP8266 publish. Nếu trạng thái khớp lệnh đang chờ trước 10 giây, service ghi SUCCESS. Nếu xác nhận đến sau hạn, hệ thống giữ bản ghi FAILED và thêm một bản ghi SUCCESS để người dùng thấy cả lần hết hạn lẫn lần thực hiện sau khi mạch kết nối lại.

```
} else if (topic.endsWith(suffix: "/status") && (value.equals(anObject: "ON") || value.equals(anObject: "OFF"))) {
    String type = topic.contains(s: "/fan/") ? "FAN" : "LIGHT";
    repository.recordReported(type, value, now);
    List<Map<String, Object>> latest = repository.latestAction(type);
    if (!retained && !latest.isEmpty() && value.equals(latest.get(index: 0).get(key: "action"))
            && value.equals(latest.get(index: 0).get(key: "desired_state"))) {
        Map<String, Object> row = latest.get(index: 0);
        long elapsed = Duration.between(Instant.parse((String) row.get(key: "created_at")), Instant.now()).toMillis();
        if (row.get(key: "status").equals(obj: "PENDING") && elapsed < 10_000) {
            repository.confirmAction(row.get(key: "id"), elapsed);
        } else if (row.get(key: "status").equals(obj: "PENDING") ||
                (row.get(key: "status").equals(obj: "FAILED") && "ESP8266 không xác nhận trong 10 giây".equals(row.get(key: "result")))) {
            if (row.get(key: "status").equals(obj: "PENDING")) repository.timeoutAction(row.get(key: "id"));
            repository.recordLateSuccess(row, now, elapsed);
        }
    }
```

Hình 16 - TelemetryService đối soát phản hồi từ ESP8266

5.3 Lưu trữ và xác thực

SQLite có sáu bảng như Hình 2. Repository tạo bảng khi ứng dụng khởi động, bật kiểm tra khóa ngoại trên mỗi kết nối và lưu thời gian dạng chuỗi UTC. Truy vấn lịch sử dùng LEFT JOIN user để không ẩn lệnh cũ nếu tài khoản gửi lệnh không còn tồn tại.

```
public List<Map<String, Object>> actions() {
    return db.queryForList(
            "SELECT h.id,COALESCE(u.username,'Tài khoản đã xóa') AS username,a.name AS actuatorName,a.type AS actuatorType,h.action,h.status,h.response_time
}

public Map<String, Object> profile(int id) {
    return db.queryForMap("SELECT id,printf('USR%03d',id) AS code,username,email,full_name AS fullName,role FROM user WHERE id=?", id);
}
```

Hình 17 - SmartRoomRepository truy vấn lịch sử điều khiển

Mật khẩu được băm BCrypt. Backend giữ userId và CSRF token trong HttpSession; trình duyệt gửi cookie phiên và header X-SR-CSRF với các yêu cầu thay đổi dữ liệu. Khi tài khoản bị xóa, Backend hủy phiên cũ và trả HTTP 401.

5.4 Kiểm tra API

Bộ Postman ở cuối báo cáo có sẵn các yêu cầu đăng nhập, điều khiển LED, xem cảm biến và lịch sử. Cần đăng nhập trước khi thử các API yêu cầu phiên.

CHƯƠNG 6. THIẾT KẾ VÀ TÀI LIỆU API

Frontend hiện dùng GET /api/state cho Dashboard và hai trang lịch sử. Các endpoint chuyên biệt có thể gọi qua Postman. API điều khiển nhận lệnh bất đồng bộ: HTTP 202 chỉ có nghĩa Backend đã nhận yêu cầu; trạng thái SUCCESS phụ thuộc phản hồi MQTT từ ESP8266.

Bảng dưới đây liệt kê các route triển khai trong ApiController. Các route có dữ liệu riêng yêu cầu cookie phiên hợp lệ; POST/PUT yêu cầu thêm X-SR-CSRF.

6.1 Danh sách API

Phương thức	Endpoint	Chức năng
POST	/api/auth/register	Đăng ký tài khoản
POST	/api/auth/login	Đăng nhập, nhận CSRF token

Phương thức	Endpoint	Chức năng
POST	/api/auth/logout	Đăng xuất
GET	/api/auth/me	Hồ sơ phiên hiện tại
PUT	/api/profile	Sửa hồ sơ
PUT	/api/profile/password	Đổi mật khẩu
GET	/api/state	Dữ liệu tổng hợp cho giao diện
GET	/api/sensors/latest	Số đo mới nhất
GET	/api/sensors/history	Lịch sử cảm biến, lọc, phân trang
GET	/api/actuators	Danh sách thiết bị
GET	/api/actuators/{id}	Chi tiết thiết bị
PUT	/api/actuators/{id}/state	Điều khiển theo ID, HTTP 202
POST	/api/devices/{type}	Điều khiển LIGHT/FAN, HTTP 202
GET	/api/iot-node/status	Trạng thái kết nối ESP8266
GET	/api/actions	Lịch sử lệnh, lọc, phân trang

Bảng 11 - Danh sách các API

6.2 Mô tả các API

POST /api/auth/register nhận username, email và password, kiểm tra dữ liệu rồi lưu mật khẩu BCrypt.

POST /api/auth/login nhận username và password; trả user, token CSRF và cookie phiên HttpOnly.

GET /api/state trả sensors, actions, devices và online. Frontend lấy mẫu mới nhất, lọc và phân trang trên trình duyệt.

POST /api/devices/{type} nhận {"state":"ON"} hoặc {"state":"OFF"}; type là LIGHT hoặc FAN. Backend trả HTTP 202 với {"actionId": ..., "status":"PENDING"}.

PUT /api/actuators/{id}/state là đường dẫn điều khiển tương đương khi biết ID thiết bị. Trạng thái xác nhận đọc lại qua GET /api/state hoặc GET /api/actions.

GET /api/sensors/history hỗ trợ searchType, keyword, page, limit. GET /api/actions hỗ trợ actuator, action, status, time, date, page, limit.

Khi lệnh không được ESP8266 xác nhận trong 10 giây, status là FAILED. Phản hồi đến muộn được ghi thêm một dòng SUCCESS. Dữ liệu ánh sáng dùng đơn vị ADC.

Mã lỗi chính: 400 dữ liệu sai, 401 chưa đăng nhập hoặc phiên hết hiệu lực, 403 CSRF sai, 404 thiết bị không tồn tại, 409 đang có lệnh PENDING, 503 broker không nhận lệnh.

6.3 Luồng giao tiếp API và MQTT

Web gọi REST API trên cùng server. Backend publish lệnh tới smartroom/{fan,light}/control và subscribe các topic sensor/status của ESP8266.

Người dùng → Frontend → Spring Boot → Mosquitto → ESP8266; số đo và phản hồi quay về Spring Boot → SQLite → Frontend.

Broker giữ lệnh điều khiển gần nhất bằng retained message. Khi ESP8266 kết nối lại, mạch nhận lệnh và gửi trạng thái thực tế; Backend ghi kết quả vào action_history.

CHƯƠNG 7. TÀI LIỆU VÀ MÃ NGUỒN

Các liên kết sau dùng để xem mã nguồn, thiết kế giao diện và thử API. API local chỉ hoạt động khi máy chạy Backend, broker MQTT và ESP8266.

Mã nguồn GitHub: https://github.com/edamame69/smartroom-iot

Giao diện Figma:
https://www.figma.com/design/rwKJ8mWRWcIMwxa3AAYK8g/ESP8266-IoT-Dashboard?node-id=87-31

Tài liệu API Postman:
https://documenter.getpostman.com/view/60095257/2sBYHYvdmt